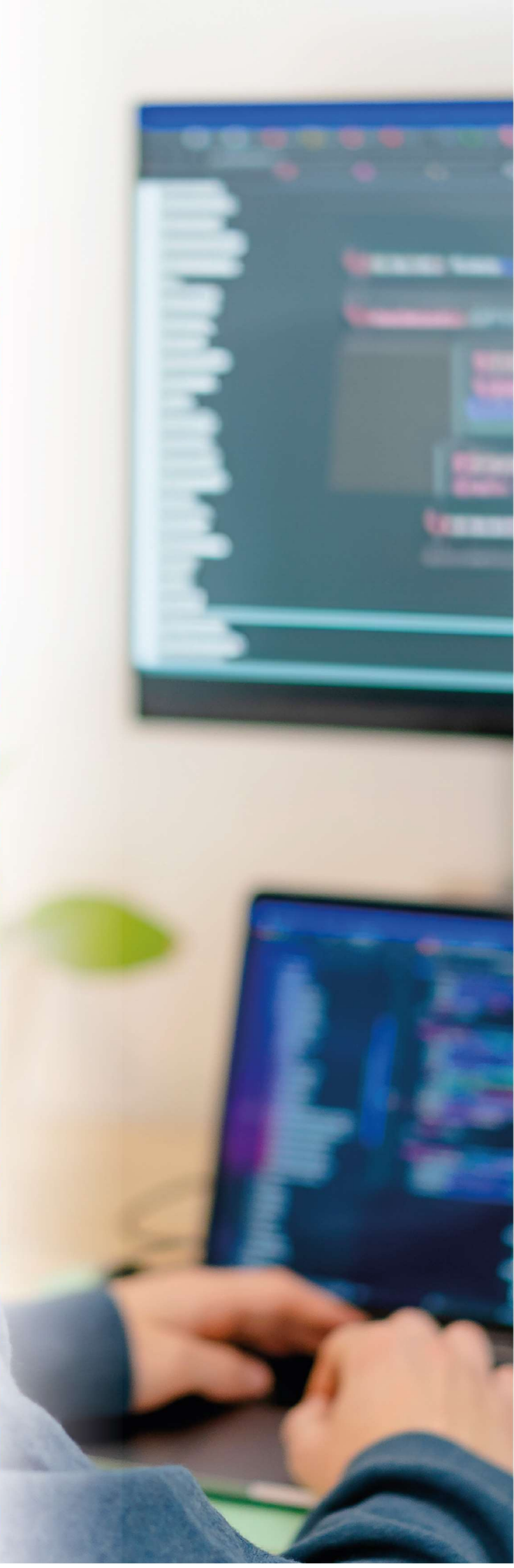
DESARROLLADOR DE APLICACIONES



Técnico Superior en Desarrollo de Aplicaciones Multiplataforma

0484 MP Bases de Datos
UD2 Bases de datos relacionales

Índice

01. Modelos de datos

02. Modelo Entidad Relación (MER)

2.1. Entidad
2.2. Relación
2.3. Atributos

03. Modelo Entidad Relación Ampliado

04. Modelo Relacional

4.1. Terminología en el modelo relacional
4.2. Restricciones en el modelo
4.3. Vistas

05. Paso del Modelo Entidad Relación al modelo relacional.

5.1. Transformación de entidades y atributos
5.2. Transformación de relaciones

06. Normalización

6.1. Dependencias funcionales
6.2. Pasos a seguir en la normalización

07. Lenguajes de datos

08. Bibliografía

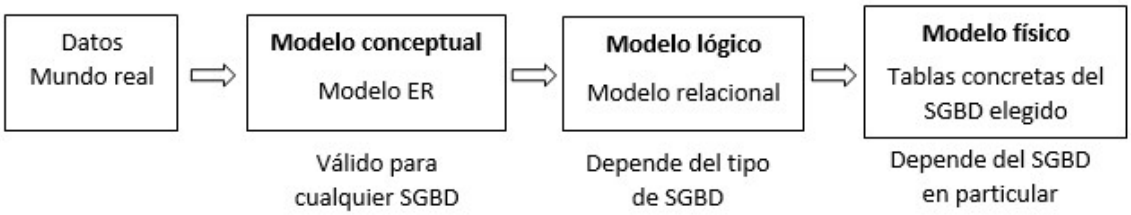
01. Modelos de datos

El diseño de una base de datos consiste en extraer todos los datos relevantes que almacenará la base de datos de mi sistema, es decir, es el proceso en el que se determina la organización de una base de datos, incluidos su estructura y contenidos.

Para obtener todos estos datos es imprescindible realizar un análisis exhaustivo del sistema que voy a modelar, es decir tendré qué determinar qué datos son necesarios guardar, descartando los que no lo son. Esta información se obtiene en reuniones con los futuros usuarios del sistema, ya que la persona que realizará el diseño de la base de datos es un informático, y, por lo tanto, no tiene por qué conocer los pormenores del negocio donde se usará la base de datos (economía, deportes, medicina, etc.).

Para realizar el diseño de la base de datos se recurre a tres modelos:

- **Modelo conceptual**. Este modelo permite representar los datos que se han de almacenar en la base de datos tal y como el usuario final, que no es un experto informático, lo concibe. Existen varios modelos conceptuales, pero estudiaremos el más extendido, el modelo Entidad/Relación (MER o Modelo ER).
- **Modelo lógico**. Los conceptos expresados en este modelo son menos entendibles para los usuarios finales, pero tienen una traducción directa al modelo físico que ya entiende el SGBD. El modelo lógico que se utilice depende del tipo de bases de datos, por ello existen diferentes modelos lógicos: modelo jerárquico, modelo relacional, modelo orientado a objetos, etc. Nosotros estudiaremos el modelo relacional ya que es el más extendido hoy en día.
- **Modelo físico**: este modelo será distinto según el SGBD a utiliza y generalmente se expresa mediante un lenguaje de programación de bases de datos tipo SQL. En el tema 4 aprenderemos a transformar el modelo



relacional en un modelo físico a través del DDL (lenguaje de definición de datos) de SQL.

En definitiva, en primer lugar, realizaremos el Modelo Entidad Relación (MER) para representar la realidad que queremos transformar en base de datos. A partir de este MER, realizaremos el Modelo relacional, el cual opera directamente con la información que formará la base de datos.

02. Modelo Entidad Relación (MER)

El objetivo del MER es identificar y representar las cosas de importancia en el funcionamiento del negocio que estamos modelando (entidades), sus propiedades (atributos), y la forma en que estas entidades se relacionan entre sí.

Este modelo fue propuesto por Peter Chen en 1977 y, a pesar del tiempo transcurrido, sigue siendo un modelo de plena actualidad gracias a su sencillez. Esta sencillez permite representar los datos de manera que un usuario final, que no es experto informático, pueda validar que el modelo propuesto es lo que se desea.

El MER se basa en la representación de la información en 3 categorías:

- **Entidades**: objetos que se van a modelas.
- **Atributos**: propiedades de esos objetos.
- **Relaciones**: asociaciones entre entidades.

2.1. Entidad

Una entidad es un objeto acerca del cual se desea almacenar información. Por ejemplo, si estamos diseñando la base de datos de una universidad las entidades que podemos identificar son alumnos, asignaturas, profesores, etc.

Se denominan con sustantivos o nombres y su representación gráfica es un rectángulo etiquetado con el nombre de la entidad.

Una **ocurrencia de una entidad** es una instancia de esa entidad, es decir es una unidad del conjunto que representa la entidad. Por ejemplo, en la entidad asignaturas una ocurrencia en concreto sería Bases de datos, otra ocurrencia sería Programación, otra ocurrencia sería Sistemas Informáticos, etc.

Se pueden distinguir dos tipos de entidades, entidades fuertes y entidades débiles:

- **Entidad fuerte**: es una entidad que no depende de ninguna otra entidad, existe por sí misma.
- **Entidad débil**: es una entidad cuya existencia depende de la existencia de otra entidad que llamamos fuerte. Se representan normalmente en un rectángulo doble.

Por ejemplo, en la base de datos de una tienda se puede representar la información de un pedido con dos entidades: pedido y línea de pedido.

- Pedido representa la información genérica como el número de pedido, la fecha en la que se realizó el pedido, la fecha en que se enviaron los productos, etc.
- Línea de pedido representa la información específica sobre cada artículo pedido y la cantidad pedida de ese artículo.

Si se elimina el pedido no tiene sentido mantener la información específica sobre cada artículo pedido, por lo que habrá que eliminar todas las líneas de pedido. Por ello, se dice que línea de pedido es una entidad débil.

2.2. Relación

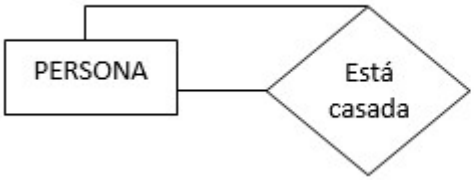
Una relación es una asociación entre los objetos representados por las entidades. Cada relación tiene un nombre que describe la asociación entre las entidades, siendo ese nombre de la relación, generalmente, un verbo. Las relaciones se representan con un rombo con el nombre de la relación y unido a las entidades que asocia.

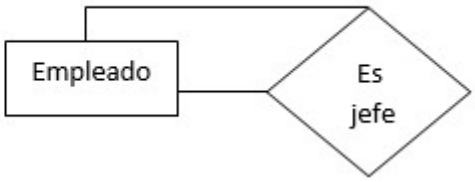
Por ejemplo, la entidad alumno puede estar relacionada con la entidad asignatura por medio de la relación cursa.

Grado de una relación:

El grado de una relación es el número de entidades que participan en esa relación. De esta manera tenemos:

- **Relaciones reflexivas**: en donde una misma entidad participa más de una vez en la relación. Por ejemplo, una persona está casada con otra persona, o un empleado es jefe de otro empleado, etc.





- **Relaciones binarias**: donde participan 2 entidades. Por ejemplo, la relación cursa vista anteriormente entre alumno y profesor, o la relación imparte entre profesor y asignatura.

- **Relaciones ternarias**, cuaternarias, etc.: donde participan 3, 4,… entidades en la relación respectivamente. Muchas veces estas relaciones complejas se pueden representar en varias relaciones binarias, simplificando con ello el sistema.

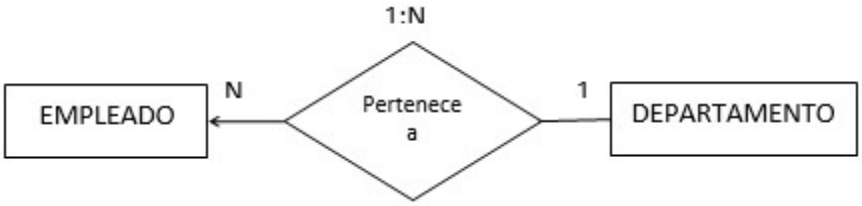
Cardinalidad de una relación:

La cardinalidad de una relación es el número máximo de ocurrencias de cada entidad que pueden intervenir en una ocurrencia de una relación. Pueden ser de tres tipos: uno a uno (1:1), uno a muchos (1:N) y muchos a muchos (N:N).

- **Uno a uno (1:1):** Cada elemento de la primera entidad está relacionada con un solo elemento de la segunda entidad y viceversa.

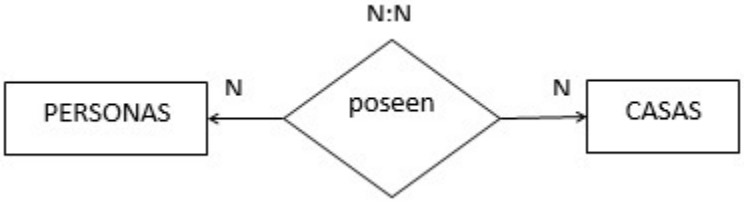
Por ejemplo: hombre está casado con mujer. La relación estar casado es una relación 1:1 ya que un hombre está casado como mucho con una mujer y viceversa, una mujer está casada como mucho con un hombre.

- **Uno a muchos (1:N):** Cada elemento de la primera entidad (A) está relacionada con uno o varios elementos de la segunda entidad (B) y cada elemento de la segunda entidad (B) está relacionado con sólo un elemento de la primera (A).
  Por ejemplo: empleado pertenece a departamento. La relación pertenece es una relación 1:N ya que un empleado pertenece a un solo departamento, pero en un departamento hay muchos empleados.



Como se observa en la representación del ejemplo en la parte N de la relación se debe poner una punta de flecha.

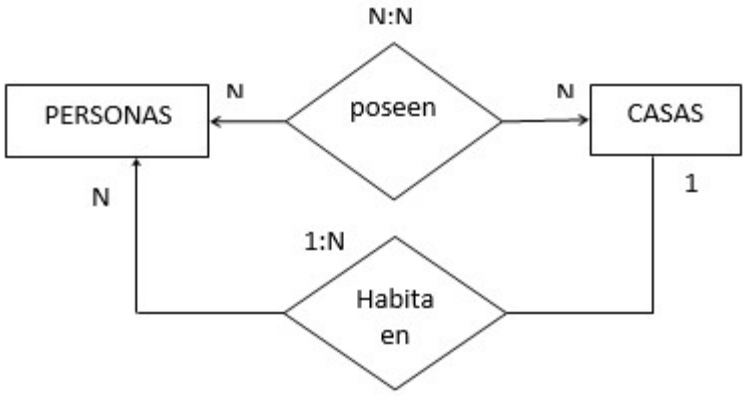
- **Muchos a muchos (N:N):** Cada elemento de la primera entidad (A) está relacionada con uno o varios elementos de la segunda entidad (B) y viceversa.
  Al igual que en la relación 1:N, la parte N de la relación (en este caso, las dos partes) se debe poner una punta de flecha.
  Por ejemplo: persona posee casas. La relación posee es una relación N:N ya que una persona puede tener varias casas y una casa puede ser propiedad de varias personas.



Ejercicio: ¿Qué cardinalidad tienen las siguientes relaciones?

- Persona habita Casa.
- Alumno cursa Asignatura.
- Profesor imparte Asignatura.
- Hombre está casado con Mujer (en monogamia).
- Hombre está casado con Mujer (en poligamia).
- Piezas forman Producto.
- Pescador pesca Pez.
- Turista viaja Hotel.
- Jugador juega en Equipo.

Cabe destacar que dos entidades pueden tener más de una interrelación. Por ejemplo, entre las entidades persona y casa pueden existir dos relaciones, la relación habita y la relación posee.



2.3. Atributos

Una entidad se representa o describe por un conjunto de atributos o propiedades. Un atributo es por tanto un dato específico, significativo para una entidad, que la califica (por ejemplo, el color), o la identifica (por ejemplo, el DNI), o la clasifica (por ejemplo, el grupo), o la cuantifica (por ejemplo el peso), o expresa su estado (por ejemplo pagado, solicitado).

Un atributo es una unidad básica de información acerca de una entidad o de una relación, es decir, no sólo las entidades tienen atributos, sino que las relaciones pueden también tenerlos.

Por ejemplo:

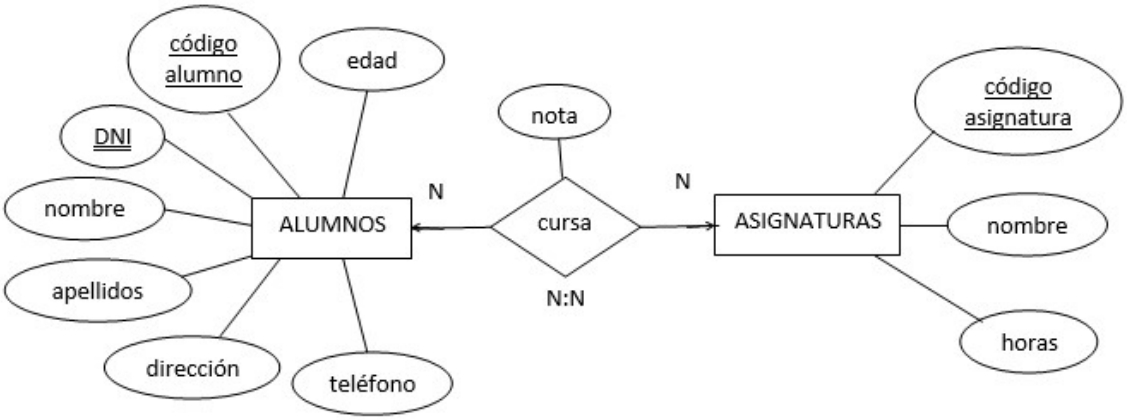
- Atributos de la entidad Alumno pueden ser: código de alumno, DNI, nombre, apellidos, dirección, teléfono, edad, email.
- Atributos de la entidad Asignatura pueden ser: código de asignatura, nombre, horas.
- Atributos de la relación cursa sería: nota (existe una nota de cada alumno por cada asignatura, por lo que nota no puede ser atributo de alumno, ya que dé a cuál de todas las asignaturas pertenece esa nota, ni puede ser un atributo de asignatura, ya que a qué alumno pertenece, por lo tanto, es un atributo de nota).

El atributo debe tener un nombre en singular y único para cada entidad. Por ejemplo, no puede haber dos atributos nombre para la entidad alumno, pero si puede haber un atributo nombre en alumno y otro atributo nombre en profesor.

Para cada atributo existe un rango de valores permitidos, es lo que llamamos el **dominio del atributo**. Por ejemplo, el dominio del atributo nombre es una cadena de texto, el dominio del atributo edad es un número entero, el dominio de un atributo precio es un número decimal, etc.

De todos los atributos de una entidad, uno de ellos debe identificar de forma única un individuo del resto. Este atributo se denomina **clave**. Si hay más de una clave una será la **principal** y otras serán **alternativas**.

Por ejemplo, la clave de la entidad Alumno sería el código de alumno y el DNI ya que con cualquiera de estos datos puedo saber a qué ocurrencia de alumno en concreto me estoy refiriendo. De entre las dos claves, puedo escoger, por ejemplo, código alumno como clave primaria y, por lo tanto, el DNI será la clave alternativa.



Los atributos se representan gráficamente mediante elipses conectadas directamente con la entidad. Además, la clave principal se indica subrayando su nombre, y las claves alternativas también se subrayarán, pero con una doble línea.

Tipos de atributos:

- **Atributos opcionales**: atributos que pueden no tomar un valor porque sea desconocido en un momento determinado. En este caso, se dice que el atributo tiene un valor nulo.
- **Atributos obligatorios**: necesariamente deben tener un valor para cada ocurrencia de entidad. Nunca puede tomar el valor nulo.
- **Atributos multivaluados**: para una misma ocurrencia de entidad pueden tomar varios valores. Por ejemplo, una persona puede tener más de un teléfono (móvil, fijo).
- **Atributos univaluados**: toman un único valor para cada ocurrencia.
- **Atributos derivados**: son aquellos cuyo valor se calcula a través de otros atributos. Por ejemplo, el atributo edad de la entidad persona se puede calcular a partir del atributo fecha nacimiento.

03. Modelo Entidad Relación Ampliado

Hay circunstancias en las que es necesario dividir una entidad en varios subconjuntos. Normalmente esta situación viene dada por dos motivos:

- Sólo ciertas ocurrencias de la entidad que cumplen una determinada condición se relacionan con otra entidad.
- Sólo ciertas ocurrencias de la entidad que cumplen una determinada condición tienen ciertos atributos innecesarios para el resto.

Para refleja esto en el diseño se crea las jerarquías. El concepto de jerarquía es introducido en el modelo Entidad Relación posteriormente, y es por eso por lo que se llama a partir de ese momento Modelo Entidad Relación Ampliado o Extendido.

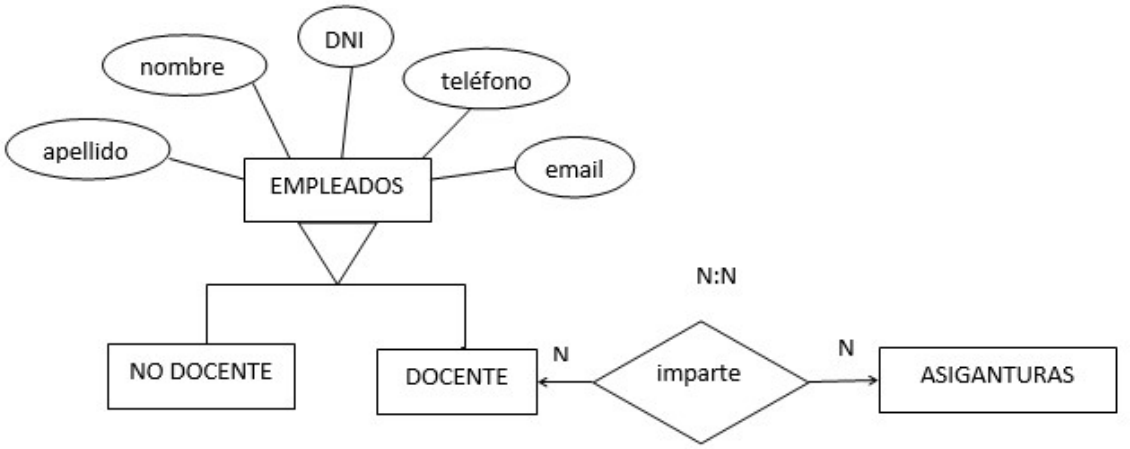
Las jerarquías consisten básicamente en una entidad llamada supertipo que agrupa los atributos y relaciones comunes a los distintos subtipos, que a su vez son versiones reducidas del supertipo con sus propios atributos y relaciones.

Las jerarquías se pueden formar de dos modos:

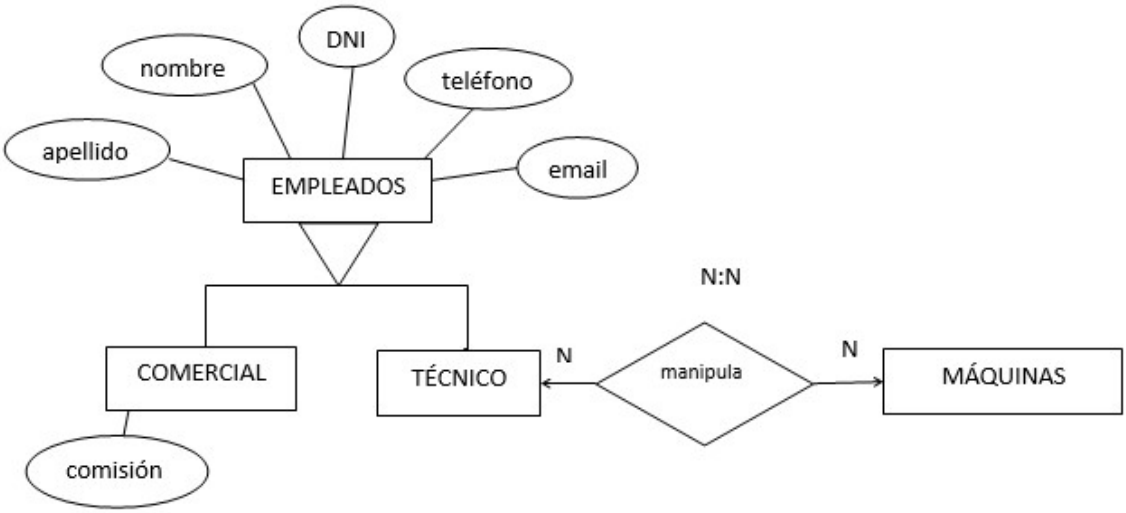
- Por **generalización**: varias entidades compartes atributos y/o relaciones, por lo que se decide crear una entidad supertipo que agrupe todas esas características comunes.
- Por **especialización**: varias ocurrencias se diferencian en algún atributo y/o relación del resto, por lo que se decide crear varias entidades subtipo para reflejar esto.

La representación gráfica de las jerarquías consiste en un triángulo pegado por la base a la entidad supertipo.

Ejemplo: Los empleados de una universidad pueden clasificarse en dos tipos, docentes y no docentes, evidentemente, como su propio nombre indica, sólo el personal docente impartirá clase. Si suponemos que una asignatura puede ser impartida por varios profesores, la cardinalidad de la relación imparte es N:N, en el caso ninguna asignatura pueda tener más de un profesor, la cardinalidad sería 1:N.



Ejemplo: Los empleados de empresa pueden clasificarse en técnicos y comerciales. De todos los empleados se almacena la misma información (nombre, apellido, DNI, teléfono, email), pero de los comerciales se debe almacenar además la comisión que reciben. Además, solamente los empleados que son técnicos trabajan con ciertas máquinas.



Ejercicio 1: Realiza el Modelo Entidad Relación para el siguiente supuesto:
Una empresa guarda información sobre sus empleados y sus departamentos. De los departamentos se guarda el código y nombre. De los empleados se conoce el código, DNI y nombre, el departamento al que pertenece, la fecha de alta en la empresa y la fecha de alta en dicho departamento.
Interesa conocer el empleado que es jefe de cada departamento. Un empleado es jefe de un sólo departamento y un departamento tiene un solo jefe.
La empresa desarrolla proyectos. De los proyectos se conoce su código, nombre y fecha de inicio. Los empleados trabajan en el desarrollo de los proyectos. Interesa conocer cuantas horas han destinado a cada proyecto. Un empleado puede trabajar en muchos proyectos y viceversa.

04. Modelo Relacional

Como se explicó al principio del tema, el diseño lógico es la etapa del diseño de una base de datos en la que se obtiene la representación de la estructura de la base de datos en términos menos entendibles para los usuarios finales, pero con una traducción directa al modelo físico que ya entiende el SGBD.

Existen muchos tipos de modelos lógicos, pero el más ampliamente usado es el modelo relacional que se estudia en este apartado.

El modelo relacional fue propuesto por Edgar Frank Codd, trabajador de IBM, en 1970, y está basado en la teoría de las relaciones.

Codd introduce el concepto de **relación** (o lo que es lo mismo **tabla**) como estructura básica del modelo. De esta manera, todos los datos de una base de datos relacional se representan en forma de relaciones (tablas) cuyo contenido va a variar en el tiempo.

En la terminología relacional, **una relación se considera un conjunto de tuplas**, o lo que es lo mismo filas.

4.1. Terminología en el modelo relacional

Relación:
La relación es el elemento básico y principal del modelo relacional. Se representa en forma de tabla, y es por ello que se utiliza indistintamente el término **relación** o **tabla**.

Una relación está formada por un conjunto de atributos (columnas) que representan una propiedad o característica de la misma.

Por ejemplo, una relación alumnos está formada por los atributos código de alumno, nombre, apellido, DNI, teléfono, email y fecha de nacimiento. Está relación se puede representar en forma de tabla como se muestra a continuación.

Código alumno	Nombre	Apellido	DNI	Teléfono	Email	Fecha nacimiento
1	Ana	Sánchez	775544	654321123	ana@gmail.com	11/08/1980
2	Pedro	Díaz	558877	666555444	pedro@gmail.com	04/07/1995
3	Javier	Alonso	665544	665553332	javier@gmail.com	06/08/1990
4	Domingo	Pérez	442233	633322211	domingo@gmail.com	01/02/1987

Las **ocurrencias de la relación** son cada una de las filas (también llamadas tuplas en este modelo). En nuestro ejemplo, la alumna Ana Sánchez con código de alumno 1, DNI 775544, teléfono 654321123, email ana@gmail.com y fecha de nacimiento 11/08/1980, sería una de las ocurrencias de la relación.

Atributos:
Un atributo es una característica que describe una relación y que toma valores en un determinado dominio.

Cada atributo puede tomar ciertos valores definidos en un **dominio**, por lo tanto, se puede definir dominio como el conjunto de valores permitidos para un atributo.

Por ejemplo, el atributo fecha de nacimiento de nuestra relación alumno, toma valor en el dominio de las fechas, el atributo nombre toma valor en el dominio de los caracteres alfanuméricos, el código del alumno en el dominio de los números enteros, etc.

El valor de un atributo para una fila puede no conocerse en un momento dado, en cuyo caso toma el valor nulo. Ojo, no hay que confundir un **valor nulo** con un valor numérico igual a cero o una cadena de caracteres vacíos.

El hecho de que una relación se represente en forma de tabla es la cause que se utilicen los términos:

- **Tabla**, en lugar de **relación**.
- **Fila**, en lugar de **tupla**.
- **Columna**, en lugar de **atributos** o **campos**.

Claves:
En una relación podemos diferenciar distintos tipos de claves:

- **Clave candidata**: es un conjunto de atributos de identifica de manera unívoca cada ocurrencia de la relación. Una relación puede tener varias clave candidatas y una clave candidata puede estar formada por más de un atributo.
  Por ejemplo, en la relación alumno, código de alumno y DNI son dos claves candidatas. Sin embargo, la unión de DNI y email no sería clave candidata, ya que no necesitamos el email para identificar a un alumno, con el DNI es suficiente.

- **Clave primaria**: de entre todas las claves candidatas identificadas anteriormente, el diseñador selecciona una, la que él quiera, como clave primaria. Ojo, la clave primaria no puede tomar valores nulos, si la clave primara es compuesta (está formada por varios atributos) ninguno de ellos

puede tomar el valor nulo. Cabe destacar que es obligatorio que todas las relaciones tengan una clave primaria.
Por ejemplo, en la relación alumno, podemos escoger código de alumno como clave primaria.

- **Clave alternativa o secundaria**: son las claves candidatas que no han sido seleccionadas como clave primaria.
  Por ejemplo, DNI será clave alternativa de la relación alumno.

- **Clave ajena**: Un atributo de una tabla es clave ajena (*foreing key*) si es clave principal en otra tabla, es decir, es el conjunto de atributos de una relación cuyos valores coinciden con los valores de la clave primaria de otra relación (pudiendo ser la misma relación). La clave ajena y la clave primara deben estar definidas obligatoriamente sobe los mismos dominios.
  Por ejemplo, tenemos la relación profesor y asignatura con la siguiente estructura:

  profesor (DNI, nombre, apellido, email, teléfono)
  asignatura (código asignatura, nombre, horas, profesor)

  El atributo DNI es la clave primaria en la tabla profesor, y el atributo código asignatura es clave primaria de la tabla asignatura.
  Si en nuestra universidad una asignatura es impartida por un solo profesor (es decir no hay asignaturas que sean impartidas simultáneamente por dos profesores), debe de existir en la tabla asignatura un atributo profesor que referencia al profesor que imparte esa asignatura, en concreto referencia al DNI del profesor (ya que es la clave primaria en la tabla profesor). Al disponer del DNI del profesor, al ser clave primaria, automáticamente puedo acceder a todos los datos del mismo.
  Por lo tanto, el campo profesor en la tabla asignatura es una clave ajena.

4.2. Restricciones en el modelo

En el modelo relacional debemos reflejar no sólo los datos y las relaciones entre estos datos, sino también ciertas condiciones que queremos que se cumplan. Por ejemplo, en la base de datos de nuestra universidad posibles restricciones pueden ser que cada alumno esté matriculado como mínimo en una asignatura, que cada alumno tenga un código asociado obligatoriamente, que la fecha de nacimiento del alumno sea anterior a una fecha dada (obligando a que tenga mínimo 17 años), etc.

Estas condiciones que se imponen al modelo se pueden dividir en dos grupos:

- **Restricciones inherentes al modelo**: estas restricciones no dependen del diseñador, ya que son derivadas de la propia definición del modelo relacional.
- **Restricciones de usuario**: en función de los requisitos a modelar, hay una serie de restricciones que son impuestas por el diseñador.

Restricciones inherentes al modelo relacional:

Estas restricciones, que no tienen que ser definidas por el diseñador ya que se derivan de la propia estructura del modelo relacional, son:

- **No puede haber dos tuplas (filas) iguales**, lo que significa que no puede haber dos filas u ocurrencias de una relación con el mismo valor en todos los campos. Como es obligatorio que toda relación tenga una clave primaria, la propia definición de clave primaria hace que se cumpla esta restricción.
- **El orden de las tuplas (filas) no es significativo**. No se requiere un orden determinado en las filas que forman la tabla.
- **El orden de los atributos (columnas) no es significativo**.
- **Cada atributo sólo puede tomar un único valor del dominio**, por lo tanto, no se admiten grupos repetitivos. Por ejemplo, en un atributo teléfono, no puedo insertar dos números de teléfono distinto.
- **Ningún atributo que forme parte de la clave primaria de una relación puede tomar un valor desconocido o inexistente (nulo).**
- **Integridad referencial:** Si una tabla tiene una clave ajena (que es clave principal en otra tabla) su valor en la 1ª tabla tiene que coincidir exactamente con algún valor en la 2ª tabla o ser NULL.
  En nuestro ejemplo de clave ajena significa que, el valor del campo profesor en la tabla asignatura o bien coincide con algún valor del campo DNI de la tabla profesor o sino es nulo. Esto quiere decir que no pueden estar impartiendo asignaturas profesores que no están dados de alta en nuestra base de datos.

Tabla profesores

DNI	Nombre	Apellido	Teléfono	Email	Fecha nacimiento
332244	Jorge	Martínez	669988774	jorge@ufv.es	01/08/1970
441177	Sergio	León	611223344	sergio@ufv.es	03/12/1980
887744	Carmen	García	666554477	carmen@ufv.es	15/01/1975
114423	César	López	632659874	cesar@ufv.es	01/03/1977

Tabla Asignatura

Código asignatura	Nombre	Horas	Profesor
DAM_BBDD	Bases de datos	6	887744
DAM_SI	Sistemas Informáticos	6	332244
DAM_FOL	Formación y orientación laboral	3	441177
DAM_LM	Lenguaje de marcas	4	114423
DAM_PROG	Programación	8	222222

¡ERROR!
El profesor con DNI 222222 no existe en la relación profesor

Aunque no es una restricción inherente al modelo, conviene resaltar en este punto que los valores que toman los atributos de las relaciones van cambiando con el tiempo, por lo tanto, si nuestra relación tiene alguna clave ajena debemos determinar las acciones que deben tomarse como consecuencia de operaciones de modificación y borrado realizadas sobre filas de las tablas referenciadas.
Por ejemplo, cuando eliminamos o modificamos los datos de un profesor, ¿qué ocurre con las filas de la tabla asignatura impartida por ese profesor?

Pueden darse tres posibilidades:

- **Operación restringida**: sólo se puede borrar (o modificar) una fila de la tabla que tiene la clave primaria referenciada si no existen filas con esa clave ajena en la tabla que referencia.
  En nuestro ejemplo, sólo podemos borrar o modificar un profesor (en el caso de la modificación se refiere al DNI) si no hay asignaturas impartidas por ese profesor.
- **Operación con transmisión en cascada**: el borrado o la modificación de una fila de la tabla que contiene la clave primaria lleva consigo el borrado o

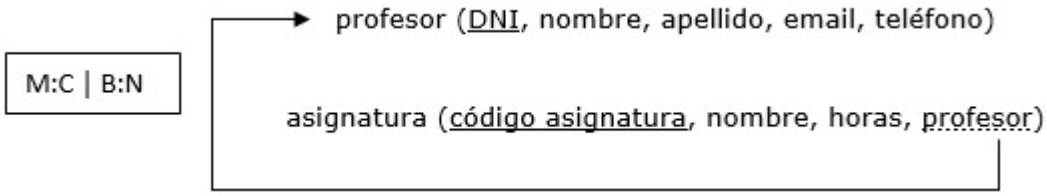
modificación en cascada de las filas de la tabla que referencia cuya clave ajena coincide con el valor de la clave primaria de la tabla referenciada. En nuestro ejemplo, el borrado o la modificación de un profesor implica la eliminación o modificación de todas las asignaturas impartidas por ese profesor.

- **Operación con puesta a nulos**: el borrado o la modificación de una fila de la tabla que contiene la clave primaria lleva consigo la puesta a nulos de los valores de la clave ajena de las filas de la tabla que referencia cuya clave ajena coincide con el valor de la clave primaria en la tabla referenciada. En nuestro ejemplo, el borrado o la modificación de un profesor implica poner a nulo los valores coincidentes de profesor en la tabla asignatura.

De todas estas opciones, la que más encaja en nuestro caso es:

- Modificación en cascada: ya que, si se modifica el DNI del profesor en la tabla profesor, lo que interesa es que se modifique ese DNI en la tabla asignatura; si se modifican el resto de los campos (nombre, apellido, email o teléfono), no afecta a la tabla asignatura.
- Borrado con puesta a nulos: ya que, si se borra un profesor, no tiene sentido que se borre la asignatura, por lo que no sería en borrado en cascada, ni tampoco tiene sentido el borrado restringido, es decir no poder borrar un profesor mientras tenga asignaturas asignadas, porque ¿y si el profesor decide dejar el centro o es despedido? Por todo ello, lo que más sentido tiene es que si se borra un profesor, las asignaturas que imparte se profesor sean puestas a nulo en su campo profesor hasta que se asigne un profesor nuevo a dicha asignatura.

Su representación gráfica sería como se muestra a continuación.



Restricciones de usuario:

Estas restricciones, que dependen del sistema que sistema que se esté modelando, son:

- **Restricciones de verificación:** estas restricciones permiten imponer condiciones a los atributos de una relación.

Por ejemplo, el campo edad de la tabla alumno debe ser mayor de 17 años, el campo sueldo de la tabla profesor debe ser mayor de 1000€, etc.

- **Disparadores:** Son procedimientos programados por el usuario que se ejecutan para hacer una tarea concreta en el momento de insertar modificar o eliminar información de una tabla.
  Por ejemplo, en una base de datos de un banco que consta de dos relaciones, una que almacena la información de cada cuenta de cada cliente, y la segunda que almacenará los movimientos realizados en cada una de las cuentas, resulta interesante programar un disparador para que en el momento en que se introduzca un nuevo movimiento en una cuenta (lo cual sería ingresar o sacar dinero), se actualice automáticamente el campo saldo de dicha cuenta en la tabla cuentas.

4.3. Vistas

Una vista de una base de datos, también llamada tabla virtual, es el resultado de una consulta sobre una o varias tablas.

Las vistas tienen la misma estructura de una tabla en el modelo relacional, es decir, están formadas por filas y columnas, pero la gran diferencia es que solamente se almacenan de ellas la definición (es decir la consulta), nunca los datos.

Aunque siempre es posible consultar los datos de una vista, existen restricciones para realizar inserciones, modificaciones o borrado de datos sobre vistas.

Por ejemplo, en la base de datos de una universidad en la que tengo alumnos que cursan asignaturas y profesores que imparten las mismas, podríamos definir una vista con algunos de los datos de los profesores (nombre, apellido, teléfono, email) que son relevantes para la jefatura de estudios, sin incluir otros datos como DNI o número de cuenta que son totalmente irrelevante para ellos.

Las vistas son importantes en el diseño de las bases de datos cumpliendo dos funciones:

- Proporcionan un mecanismo de seguridad potente y flexible al ocultar partes de la base de datos a distintos usuarios y aplicaciones.
- Permiten a los usuarios y aplicaciones acceder a los datos de forma personalizada según sus necesidades.

05. Paso del Modelo Entidad Relación al modelo relacional.

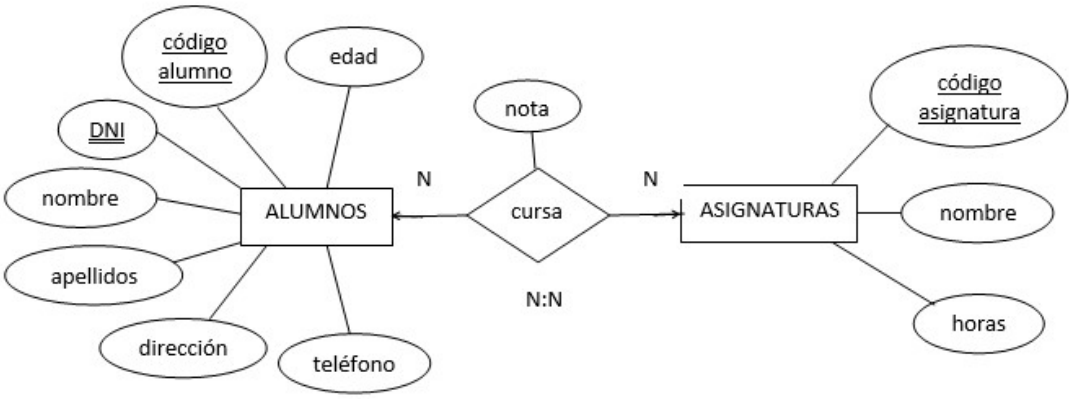
Para obtener las tablas del modelo relacional explicado en el punto anterior se aplican unas reglas de transformación al Modelo Entidad Relación (MER) creado como primer paso del diseño. Es decir, estas reglas me permitirán traducir las entidades y relaciones que forman el MER en las tablas que forman el modelo relacional.

5.1. Transformación de entidades y atributos

Cada entidad del MER se transforma en una tabla con el mismo nombre, los mismos atributos y la misma clave primaria (al igual que en el MER se subrayará con subrayado simple).

En el caso de que existan claves alternativas en el MER, seguirán siendo claves secundarias o alternativas en el modelo relacional y se indicará subrayando doble estos atributos.

Por ejemplo, si partimos del modelo entidad relación creado anteriormente en este tema en el que se modela la base de datos de una universidad donde los alumnos cursan diferentes asignaturas, las tablas que se obtendrán aplicando esta regla de transformación de entidades son:



ALUMNO (código alumno, DNI, nombre, apellidos, dirección, teléfono, edad)

ASIGNATURA (código asignatura, nombre, horas)

Los atributos opcionales del MER, es decir aquellos que pueden tomar un valor nulo, se escribirán seguidos de un asterisco.

Por ejemplo, si la edad del alumno se considera un atributo opcional, es decir, puede ser desconocido y por tanto tomar el valor nulo, esta relación se representaría:

ALUMNO (código alumno, DNI, nombre, apellidos, dirección, teléfono, edad*)

La regla general para transformar los atributos multivaluados del MER es convertir el atributo en una nueva relación o tabla cuya clave estará formada por la concatenación de la clave de la relación en la que se encuentra y el nombre del atributo.

5.2. Transformación de relaciones

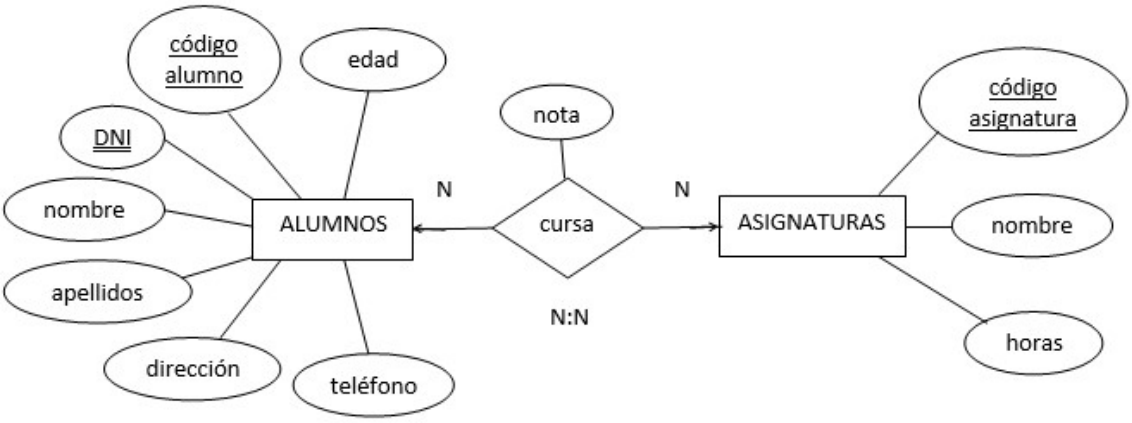
Cada relación del MER se transformará de manera distinta dependiendo de su tipo de cardinalidad.

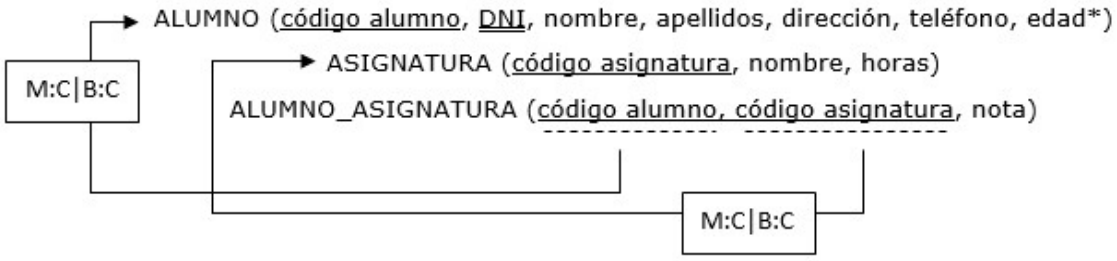
Transformación de relaciones N:N

Toda relación N:M se transforma en una tabla con sus atributos (si los tiene), de tal manera que:

- La nueva tabla tiene como clave primaria, las claves de las entidades que relaciona.
- Además, cada elemento de la clave compuesta es clave ajena en las tablas obtenidas de las entidades asociadas por dicha relación.

En nuestro ejemplo, además de las tablas alumnos y asignatura, aplicando esta regla de transformación, añadiríamos una tabla que podemos llamar cursa o alumno_asignatura de la siguiente manera.





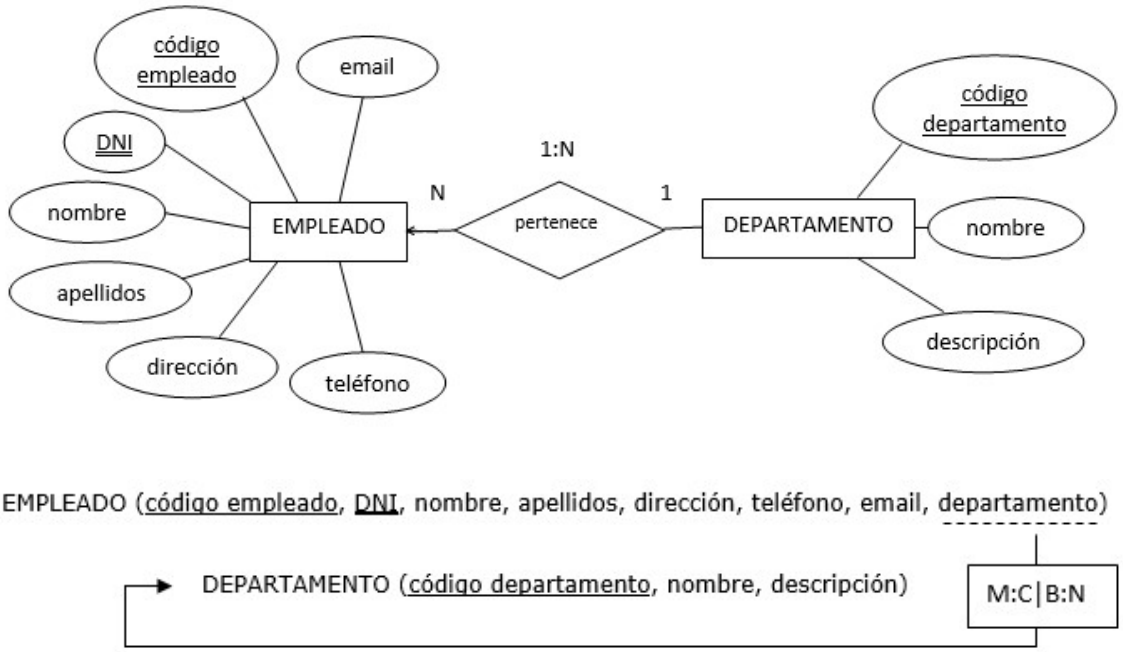
Cada una de las claves ajenas se representarán como se indica en el ejemplo (clave ajena subrayada punteada y con una flecha que me dirige hacia la tabla referenciada). Además, indicaré las opciones de borrado y modificación de la siguiente manera:

- M:C | B:C → Modificación/borrado en cascada.
- M:N | B:N → Modificación/borrado con puesta a nulos.
- M:D | B:D → Modificación/borrado con valor por defecto.
- M:R | B:R → Modificación/borrado restringida.

Transformación de relaciones 1:N

Para toda relación 1:N se propagan los atributos de la clave primaria de la entidad con cardinalidad 1 a la que tiene cardinalidad N (en el sentido de la flecha de la relación). La clave propagada se transforma en clave ajena, y se deberá por tanto indicar también las opciones de modificación y borrado tal y como se ha explicado anteriormente.

Por ejemplo, la relación entre un empleado y un departamento, representado por el MER que se muestra a continuación, se transformaría en las tablas:

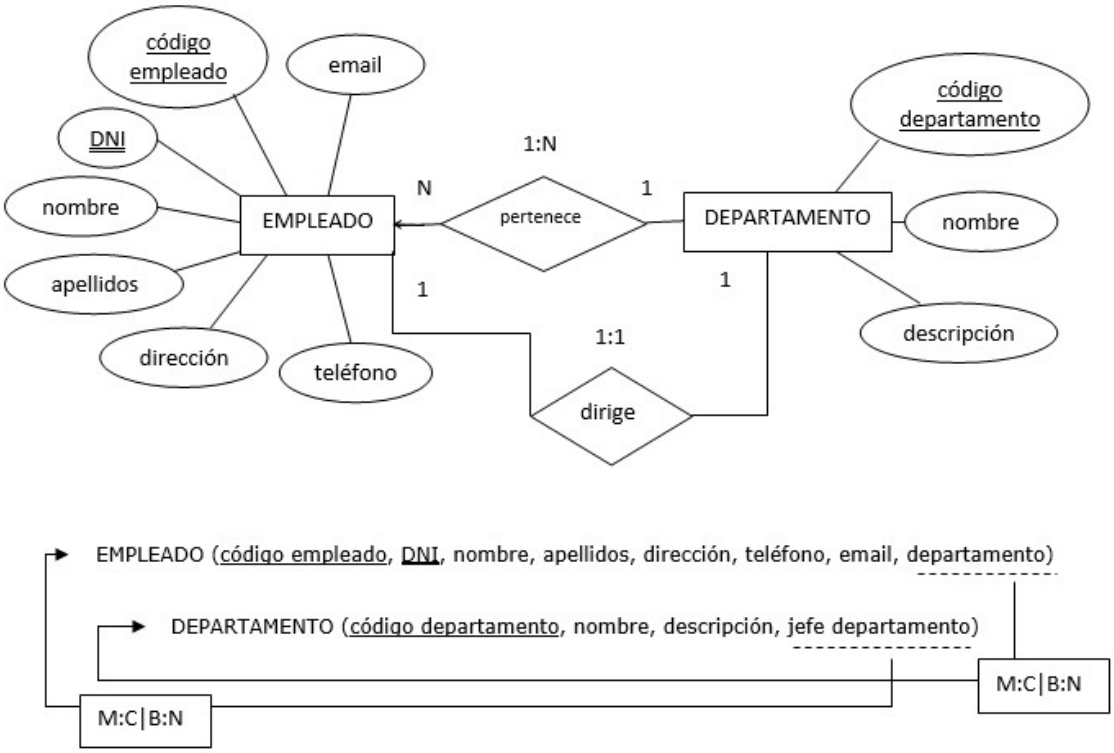


Transformación de relaciones 1:1

Para la transformación de las relaciones 1:1 se propagará la clave primaria de cualquiera de las entidades hacia la otra, teniendo en cuanta de que se genere el menor número de nulos posible. La clave propagada se transforma en clave ajena, y se deberá por tanto indicar también las opciones de modificación y borrado tal y como se ha explicado anteriormente.

Por ejemplo, entre las entidades empleado y departamento existe la relación pertenece, tal y como se ha visto anteriormente, pero también puede existir la relación dirige para indicar qué es empleado es el jefe de departamento. De esta manera el modelo entidad relación y su correspondiente transformación a tablas del modelo relacional quedaría como se muestran a continuación.

En este caso se ha decidido migrar la clave primaria de empleado a departamento ya que genera muchos menos nulos.



Transformación de jerarquías

En general se pueden dar tres posibilidades:

- **Englobar todos los atributos de la entidad y sus subtipos en una sola relación**: se llevará a cabo esta posibilidad cuando los subtipos se diferencian en muy pocos atributos y las relaciones que los asocian con el resto de las entidades del esquema sean las mismas para todos los subtipos.
- **Crear una relación para el supertipo y tantas relaciones como subtipos haya con sus atributos correspondientes**: se implementará esta solución cuando existen muchos atributos distintos entre los subtipos y se quieren mantener los atributos comunes a todos ellos en una relación.
- **Considerar relaciones distintas para cada subtipo que contengan además los atributos comunes**: en el caso de que existan muchos atributos distintos entre los subtipos (igual que en el caso anterior) y los accesos realizados sobre los datos de los distintos subtipos afecten casi siempre a atributos comunes se utilizará esta posibilidad.

Ejercicio 2: Transforma el Modelo Entidad Relación que creaste en el ejercicio 1 (página 10, empresa que almacena información sobre sus empleados, departamentos y proyectos) en su correspondiente modelo relacional.

06. Normalización

Una vez que hemos transformado el modelo entidad relación al modelo relacional como hemos hecho anteriormente, ya disponemos de las tablas que formarán mi base de datos. Es interesante comprobar la calidad de este modelo relacional, y si no cumple determinados criterios de calidad, refinar el diseño para lograr la calidad deseada.

Uno de los parámetros que mide la calidad de una base de datos es la forma normal en la que se encuentra su diseño. Esta forma normal se alcanza cumpliendo unas condiciones que impone cada forma normal al conjunto de atributos de una relación. El proceso de obligar a los atributos de una relación a cumplir ciertas formas normales se llama normalización.

El objetivo de la normalización es conseguir, basándose en las relaciones obtenidas en la transformación del modelo conceptual (MER) y aplicando unas normas concretas, un conjunto de relaciones óptimo que, una vez implementado físicamente:

- Evite anomalías en operaciones de manipulación de datos.
- Forme una estructura flexible y fácil de mantener.
- Evite redundancias superfluas.
- Aminore el espacio requerido para el almacenamiento.
- Evite problemas de integridad.

6.1. Dependencias funcionales

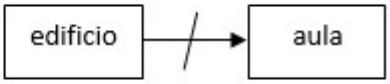
Por la importancia que tienen en el procedimiento de normalización las dependencias entre atributos, presentamos aquí algunos conceptos que serán útiles más adelante.

Dependencia funcional

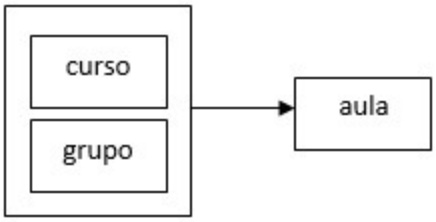
Se dice que un atributo B depende funcionalmente de otro A de la misma relación si, para todo valor A, se cumple que a cada valor de A le corresponde no más de un valor de B.

Por ejemplo, tenemos una relación alumnos con los siguientes atributos: DNI, curso, grupo, aula, edificio.

- Podemos decir que el atributo edificio depende funcionalmente del atributo aula, ya que cada aula está situada en un edificio concreto.
- El atributo aula no depende funcionalmente de edificio, ya que para cada valor de edificio existen varias aulas.



Las dependencias también se pueden expresar respecto a combinaciones de atributo. Por ejemplo, el aula depende funcionalmente de los atributos curso y grupo, ya que el primer curso del grupo de DAM siempre trabaja en un aula concreta.



Dependencia funcional completa

Se dice que un atributo C tiene dependencia funcional completa de un grupo de atributos X de la misma relación, si C depende funcionalmente de X pero no de ningún subconjunto obtenido de los posibles atributos que forman X.

Por ejemplo, el conjunto formado por curso y grupo con respecto a aula es un ejemplo de dependencia funcional completa, ya que para cada curso existen varias aulas y para cada grupo también, sólo para el par completo, el valor de aula es único.

Dependencia transitiva

Sean A, B y C tres atributos o grupos de atributos de una relación. Si B depende funcionalmente de A y C de B, pero A no depende funcionalmente de B, se dice que C depende transitivamente de A.

Por ejemplo, hay una dependencia funcional transitiva de edificio con respecto a DNI-alumno, ya que se cumple:

6.2. Pasos a seguir en la normalización

Las relaciones deben de cumplir con unas restricciones que han sido definidas en las distintas formas normales. Hay definidas 6 formas normales, cada una agrupando a las anteriores, de tal manera que, por ejemplo, la forma normal 3, cumple la forma normal 2 y la forma normal 1.

Primera forma normal (1FN)

- Definición formal: una relación está en primera forma normal si el valor de los dominios de los atributos es único, o lo que es lo mismo cada atributo es atómico.
- Definición práctica, poco rigurosa: una relación está en primera forma normal si contiene para cada intersección fila-columna un solo valor y no un conjunto de ellos.

Esta forma normal pretende garantizar la no existencia de grupos repetitivos.

Ejemplo: supongamos que queremos crear el diseño de la base de datos de una tienda de ultramarinos, donde el modelo relacional de partida está formado por una única tabla como la que se muestra a continuación. Como se puede observar a simple vista, esta tabla (o lo que es lo mismo relación) no cumple la primera forma normal (1FN).

Código pedido	Fecha pedido	Código cliente	Nombre cliente	Ciudad cliente	Tlf cliente	Código producto	Descripción producto	Precio unitario	Código proveedor	Tlf Proveedor	Cantidad pedida
001	12/3/14	32	Juan Pérez	Madrid	99999	010	Caja galletas	3€	01	444444	5
						045	Zumo piña	1€	66	555555	10
002	15/4/14	25	José López	Sevilla	11111	033	Leche	1€	66	555555	3
						095	Vino Dulce	3€	21	111111	2
						078	Champú	3€	32	222222	1

Solución:

- Una vez identificados los atributos que tienen varios valores para uno concreto de la clave, se formará con ellos una nueva relación, y se eliminarán de la antigua.
- La clave principal de la nueva relación estará formada por la concatenación de uno o varios de sus atributos y la clave principal de la antigua.

En nuestro ejemplo de la tienda de ultramarinos las tablas resultantes de aplicar estos pasos, serían las que se muestran a continuación, que ya cumplen la 1FN.

Relación Pedidos:

Código pedido	Fecha pedido	Código cliente	Nombr cliente	Ciudad cliente	Tlf cliente
001	12/3/14	32	Juan Pérez	Madrid	99999
002	15/4/14	25	José López	Sevilla	11111

Relación Líneas de pedidos:

Código pedido	**Código producto**	Descripción producto	Precio unitario	Código proveedor	Tlf Proveedor	Cantidad pedida
001	010	Caja galletas	3€	01	444444	5
001	045	Zumo piña	1€	66	555555	10
002	033	Leche	1€	66	555555	3
002	095	Vino Dulce	3€	21	111111	2
002	078	Champú	3€	32	222222	1

Segunda forma normal (2FN)

- Definición formal: una relación está en 2FN si está en 1FN y cada atributo suyo distinto de los que componen una clave candidata tienen una dependencia funcional completa de dicha clave candidata.
- Definición práctica, poco rigurosa: una relación está en 2FN si está en 1FN y cada uno de sus atributos depende de toda la clave.

Ejemplo: en el ejemplo anterior se observa que en la relación Líneas de pedidos, todos los atributos menos el atributo “cantidad pedida” no tienen dependencia

funcional completa de la clave, sino que la tienen de una parte de la clave principal, del “código del producto”.
La relación Pedidos dispone de una clave simple, por lo tanto podemos afirmar directamente que la relación pedidos se encuentra en 2FN.

Solución:

- Una vez identificados los atributos que no dependen completamente de alguna clave candidata, sino solamente de parte de ella, se formará con ellos otra relación y se quitarán de la antigua.
- La clave principal de la nueva relación estará formada por la parte de la antigua de la que dependen totalmente.

En nuestro ejemplo, la relación línea de pedidos se dividirá en dos relaciones: Líneas de pedidos y Productos, las cuales ya estarán en 2FN.

Relación Líneas de pedidos:

Código pedido	Código producto	Cantidad pedida
001	010	5
001	045	10
002	033	3
002	095	2
002	078	1

Relación Productos:

Código producto	Descripción producto	Precio unitario	Código proveedor	Tlf Proveedor
010	Caja galletas	3€	01	444444
045	Zumo piña	1€	66	555555
033	Leche	1€	66	555555
095	Vino Dulce	3€	21	111111
078	Champú	3€	32	222222

Tercera forma normal (3FN)

- Definición formal: una relación está en 3FN si está en 2FN y cada atributo que no pertenezca a una clave candidata concreta no depende transitivamente de dicha clave candidata.
- Definición práctica, poco rigurosa: una relación está en 3FN si está en 2FN y cada uno de sus atributos depende sólo de la clave.

Ejemplo: en el ejemplo, se observa que en la relación Productos, el atributo teléfono del proveedor depende transitivamente de la clave principal, que es el código de pedido, o lo que es igual, no depende funcionalmente de la clave principal, sino del atributo código de proveedor.
Así mismo, en la relación Pedidos, los atributos nombre de cliente, ciudad de cliente y teléfono de cliente dependen del atributo código de cliente.

Solución:

- Una vez identificados los atributos que dependen de otro atributo distinto de la clave candidata, se formará con ellos otra relación y se quitarán de la antigua.
- La clave principal de la nueva relación será el atributo del cual dependen. Este atributo en la relación antigua pasará a ser una clave ajena.

En nuestro ejemplo, la relación productos se dividirá en dos relaciones: Productos y Proveedores, las cuales ya estarán en 3FN. Y la relación Pedidos se dividirá en las relaciones pedidos y clientes, que estarán ya también en 3FN.

Relación Pedidos:

Código pedido	Fecha pedido	Código cliente
001	12/3/14	32
002	15/4/14	25

Relación Clientes:

Código cliente	Nombre cliente	Ciudad cliente	Tlf cliente
32	Juan Pérez	Madrid	99999
25	José López	Sevilla	11111

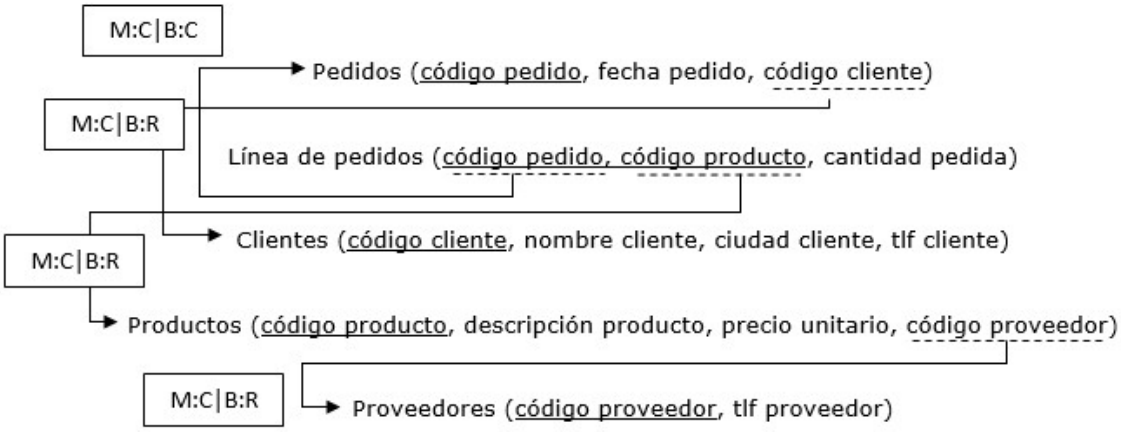
Relación Productos:

Código producto	Descripción producto	Precio unitario	Código proveedor
010	Caja galletas	3€	01
045	Zumo piña	1€	66
033	Leche	1€	66
095	Vino Dulce	3€	21
078	Champú	3€	32

Relación Proveedores:

Código proveedor	Tlf Proveedor
01	444444
66	555555
66	555555
21	111111
32	222222

Ahora podemos asegurar que este conjunto de relaciones está en 3FN, el cual es el nivel mínimo exigible en un buen diseño de una base de datos. Por lo tanto, nuestro modelo relacional quedaría de la siguiente forma:



Existen además otras 3 formas normales para un mejor diseño: forma normal de Boyce y Codd (FNBC), cuarta forma normal (4FN) y quinta forma normal (5FN). Una relación está en FNBC si está en 3FN y todo determinante (atributo que determina completamente a otro) es clave candidata. Y las forman normales 4 y 5 (4FN y 5FN) se ocupan de las dependencias entre atributos multivaluados.

07. Lenguajes de datos

Una vez obtenido el modelo relacional normalizado, debemos de tener algún modo de implementar este modelo en el SGBD. Esto nos lo permiten los lenguajes de datos, con los que podemos, definir cada elemento y restricción del sistema o manipular sus datos.

Estos lenguajes de datos los podemos clasificar según su función en:

- **Lenguaje de definición de datos (DDL)**: permite la definición de las relaciones y restricciones del modelo.
- **Lenguaje de manipulación de datos (DML)**: son los responsables de la parte dinámica del modelo ya que permiten modificar el contenido de las relaciones mediante sentencias.

- **Lenguaje de control de datos (DCL)**: es el lenguaje para el control de acceso a los datos y objetos de las bases de datos.

08. Bibliografía

Hueso Ibañez, L. (2012). Bases de datos. Editorial Ra-Ma.

López Montalbán, I. y de Castro Vázquez, M. (2014). Gestión de bases de datos. Editorial Garceta.

Date, C. J. (2001). Introducción a los sistemas de bases de datos. Editorial Pearson Prentice Hall.

De Miguel, A.; Piattini, M.; Marcos, E. (1999). Diseño de bases de datos relacionales. Editorial Ra-Ma.